

# Tiny Vision-Language Model that Spells Scenes

Technical Report – RLS Entrance Challenge

Ons Chtioui

September 29, 2026

## 1 Problem

The model must look at a 64x64 image (one or two colored shapes on a plain background) and write, letter by letter, the word describing the scene (size+color+shape, optionally followed by a spatial relation and a second object). The learning goal was to build every component of a minimal Vision-Language Model from scratch, without shortcuts: a visual encoder, hand-written multi-head attention, a causal Transformer decoder, and to honestly measure what the model truly understands about the image rather than chasing a high score alone.

## 2 Architecture

The pipeline follows exactly the schema required by the brief: image  $\rightarrow$  CNN encoder  $\rightarrow$  linear projection (adapter)  $\rightarrow$  visual tokens  $\rightarrow$  concatenation with letter tokens  $\rightarrow$  prefix-masked Transformer decoder  $\rightarrow$  linear head  $\rightarrow$  greedy letter-by-letter decoding.

| Stage         | Tensor shape   | Design choice                                                                       |
|---------------|----------------|-------------------------------------------------------------------------------------|
| Image         | (B, 3, 64, 64) | –                                                                                   |
| CNN encoder   | (B, 128, 8, 8) | 3 Conv2d layers, stride 2, channels 32-64-128, BatchNorm+ReLU                       |
| Adapter       | (B, 64, 128)   | Linear 128 $\rightarrow$ $d_{\text{model}}$ , spatial flatten into 64 visual tokens |
| Full sequence | (B, 64+L, 128) | $d_{\text{model}} = 128$ , 4 heads, 2 decoder layers, FFN=256                       |
| Logits        | (B, L+1, 27)   | $27 = 26 \text{ letters} + \text{eos}$                                              |

Table 1: Architecture pipeline summary.

**Masking choice:** A prefix mask: visual tokens attend to each other freely (bidirectional), while letter tokens are causal among themselves but always see the full set of visual tokens. This reflects the idea that the image is a fully known context at the moment each letter is generated.

## 3 Implementation

Multi-head attention was written entirely by hand (Q/K/V projections, head splitting, scaled dot-product with a  $1/\sqrt{d_k}$  factor, additive mask before softmax, head merging, output projection). Its correctness was verified against `F.scaled_dot_product_attention` on the same inputs and mask: maximum observed difference of  $6.7 \times 10^{-8}$  in float32, well below the required  $10^{-5}$  threshold.

Loss masking was a subtle point: padding positions are ignored via `ignore_index=-100` in the cross-entropy, using the id as filler on the input side. An early version had an off-by-one misalignment between logits and targets: the position right after the image predicted the word's second letter instead of its first. This bug was invisible in the training loss but made exact-match accuracy zero (0.0%). It was found by inspecting generated predictions manually, then fixed by aligning targets to the full letters+eos sequence (length  $L+1$  instead of  $L$ ).

## 4 Training Setup

## 5 Experiments

**E0 – overfit one batch (bug check):** On a single batch, the loss went from 3.4512 (step 0) to 0.0088 (step 300). Interpretation: the full pipeline is wired correctly.

| Parameter           | Value                                         |
|---------------------|-----------------------------------------------|
| Optimizer           | AdamW, lr=3e-4                                |
| Batch size          | 32                                            |
| Epochs (main model) | 10                                            |
| Hardware            | Google Colab, NVIDIA Tesla T4 GPU             |
| Seed                | Fixed seed from <code>generate_data.py</code> |
| Model parameters    | ~ 1.2M                                        |

Table 2: Training Hyperparameters.

**E1 – blind baseline:** Replacing images with zeros to test if the model depends on the image.

**S1 – required throughput benchmark:** Images/second measured at batch sizes 1, 16, 64, 256 on T4 GPU after 5 warm-up iterations with explicit `torch.cuda.synchronize()`.

**Additional test – 10 vs 15 epochs:** At 15 epochs, final validation loss (0.1968) is worse than at 10 epochs (0.0313), showing overfitting past 10 epochs.

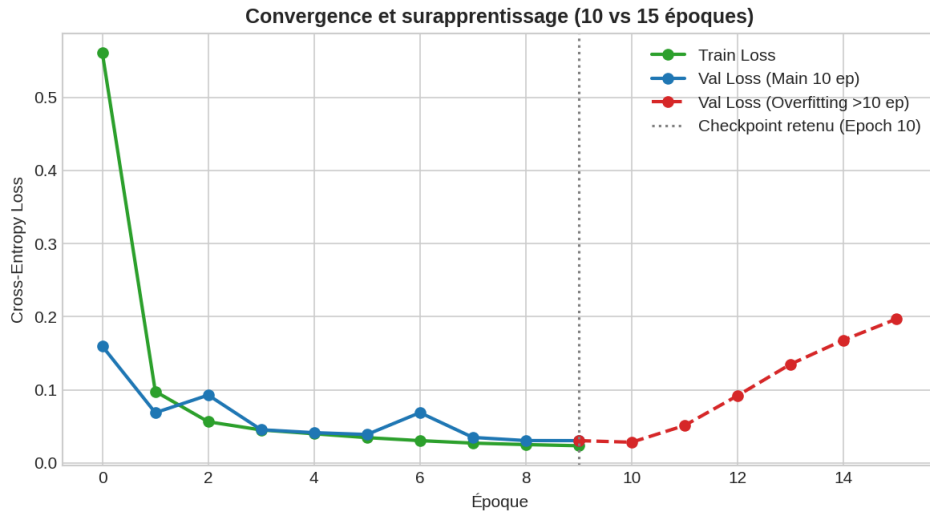


Figure 1: Training and validation loss curves showing convergence and overfitting behavior.

## 6 Results

| Experiment        | Configuration          | Metric             | Result                    |
|-------------------|------------------------|--------------------|---------------------------|
| Main model        | 10 epochs, batch 32    | Exact match (test) | 68.2%                     |
| E1 blind baseline | 10 epochs, images = 0  | Exact match (test) | 0.0%                      |
| E1 blind baseline | same                   | Final val loss     | 1.3475 (vs 0.0313 normal) |
| S1 throughput     | batch 64, Colab T4 GPU | images/s           | 5500.6                    |

Table 3: Main experiment results.

## 7 Analysis

The gap between the normal model and the blind baseline (0% vs 68.2% exact-match) shows unambiguously that the model genuinely exploits visual content. The relation attribute is the hardest to learn (41.9%), as it requires relative positional context. Attributes of the second object suffer from error accumulation during autoregressive decoding.

## 8 Limitations & Reproducibility

Single run per configuration (no variance estimate). In-memory measurements without DataLoader disk loading.

| Attribute        | Normal model | Blind baseline |
|------------------|--------------|----------------|
| size (object 1)  | 88.25%       | 49.75%         |
| color (object 1) | 80.55%       | 30.60%         |
| shape (object 1) | 81.40%       | 24.10%         |
| relation         | 41.86%       | 2.55%          |
| size (object 2)  | 75.78%       | 4.02%          |
| color (object 2) | 59.90%       | 2.25%          |
| shape (object 2) | 61.86%       | 0.98%          |

Table 4: Accuracy by attribute (normal vs blind baseline).

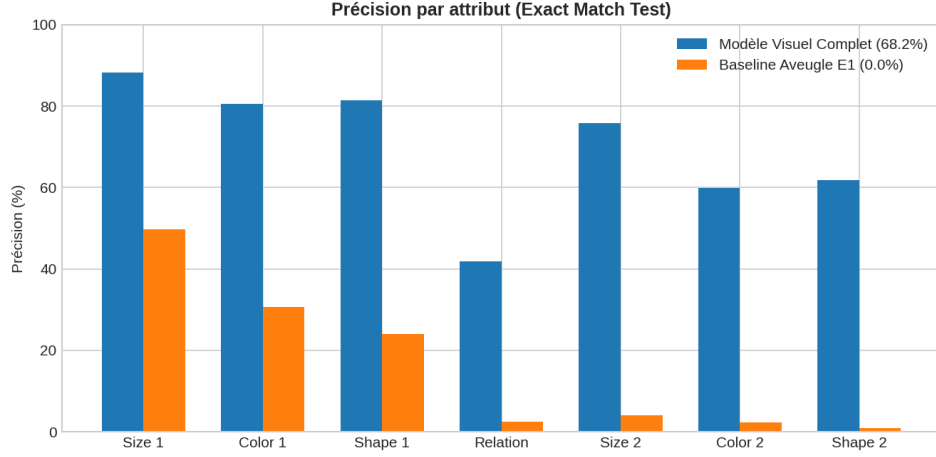


Figure 2: Per-attribute accuracy breakdown for the main model versus the blind baseline.

| Batch size    | 1     | 16     | 64     | 256    |
|---------------|-------|--------|--------|--------|
| images/second | 140.2 | 2109.0 | 5500.6 | 6804.5 |

Table 5: S1 throughput benchmark on T4 GPU.

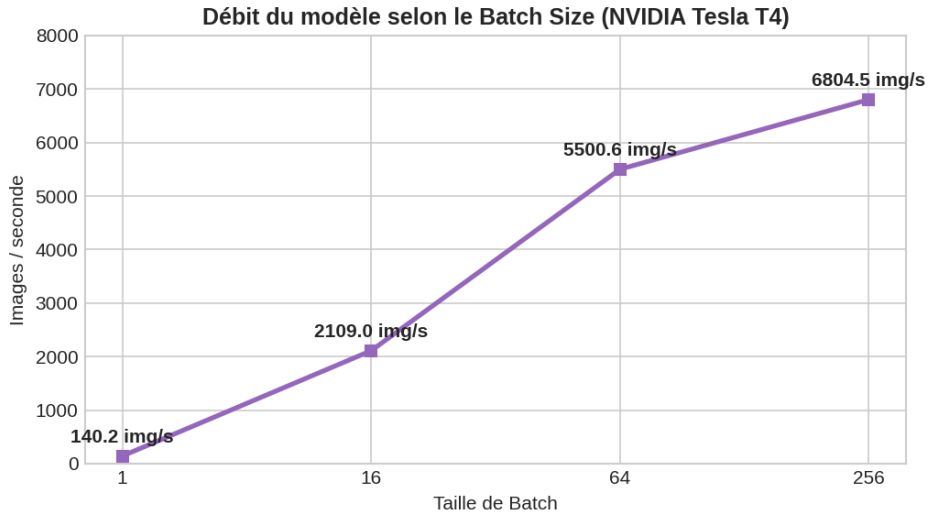


Figure 3: Throughput scaling (images/sec) across batch sizes on NVIDIA Tesla T4 GPU.

All commands to reproduce:

```
git clone https://github.com/ons-sup/rls-tiny-vlm.git && cd rls-tiny-vlm
pip install Pillow tqdm numpy && python generate_data.py
python -m tests.test_shapes && python -m tests.test_attention
python -m src.train --mode e0
python -m src.train --mode full --epochs 10
python -m src.run_eval --checkpoint checkpoint_epoch9.pt --split test
python -m src.train_blind --epochs 10
python -m src.run_eval --checkpoint checkpoint_blind_epoch9.pt --split test
python -m benchmarks.throughput
```

## 9 References

1. Vaswani et al., "Attention Is All You Need", 2017.
2. Liu et al., "Visual Instruction Tuning" (LLaVA), 2023.
3. PyTorch documentation.
4. RLS Entrance Challenge brief.